

Studio Graphene

Assessment for Full Stack Developer

Node.js + React Programme

Overview

Thank you for your interest in joining our Programme. This document contains three take-home exercises designed to help us understand how you think, code, and structure a small full-stack application.

Important: Please pick any one of the three exercises below.

You do not need to complete all three. Choose the one that interests you most and that you feel you can deliver to a high standard within the timeframe.

Time Expectation

- You have 2 to 3 days from receipt of this brief to submit your solution.
- We estimate the actual coding effort at 6 to 10 hours. Please do not feel pressured to spend more, we value quality and clarity over volume of features.
- If you run out of time, submit what you have along with a note in the README about what you would do next.

What We Are Looking For

We are evaluating you, so we are not expecting senior-engineer output. We care about:

- Clean, readable code with sensible naming and structure
- A clear separation between frontend (React) and backend (Node.js)
- Basic understanding of REST APIs, state management, and component design
- A README that explains how to run your project locally
- Honesty in the README about what works, what does not, and what you would improve with more time

Tech Stack Requirements

- **Backend:** Node.js with Express (or Fastify, if you prefer). JavaScript or TypeScript, your choice.
- **Frontend:** React (Create React App, Vite, or Next.js are all acceptable). Functional components with hooks, please, no class components.
- **Storage:** An in-memory array, a JSON file, or SQLite is fine. You do not need to set up MongoDB or PostgreSQL unless you want to.
- **Styling:** Plain CSS, CSS Modules, Tailwind, or any component library (Material UI, Chakra, shadcn/ui, etc.) of your choice.
- **Testing:** Optional but appreciated. One or two meaningful tests are better than ten trivial ones.

Submission Instructions

Please follow these steps carefully, as incomplete submissions are difficult for us to review.

1. GitHub Repository

1. Create a PUBLIC GitHub repository.
2. Push your code with a clean commit history. We will look at your commits, so please commit incrementally rather than as one giant 'final' commit.
3. Include a single repo for the whole project. A monorepo with /client and /server folders is the simplest structure and what we recommend.

2. Hosted Deployment (Recommended)

You can deploy a live version of the app.

- **Frontend:** Netlify, Vercel, or GitHub Pages are all free and easy.
- **Backend:** Render, Railway, Fly.io, or Cyclic offer free Node.js hosting. (Heroku no longer has a free tier.)
- Make sure the deployed frontend can actually talk to the deployed backend, and test it from an incognito window before submitting.

3. README.md - Required

Your README must include the following sections:

- **Project Title & Brief Description** - which exercise you chose, in one paragraph.
- **Live Demo Links** - Deployed link.
- **Tech Stack** - libraries and tools used and why.
- **How to Run Locally** - exact commands a reviewer can copy and paste. Assume they have only Node.js installed.
- **API Documentation** - list of endpoints with method, path, request body, and response shape.
- **Project Structure** - a brief tree of the folders and what lives in each.
- **Next Steps** - what you chose not to do, and what you would build next.

4. What to Send Us

Please click on the link to [submit your exercise](#) to send your work to us.

How We Will Evaluate Your Submission

To help you focus your effort, here is exactly what we will look at and the rough weighting:

Area	Weight	What we look for
Code Quality	25%	Readability, naming, file organisation, sensible abstractions
Functionality	25%	Core requirements work end-to-end without crashing
Full Stack Integration	20%	Clean REST API design, proper request/response handling, error handling
UI / UX	15%	Usable interface, loading and error states handled, responsive on mobile
Documentation, Git & Deployment	10%	Clear README, API Documentation, meaningful commits, easy setup
Bonus / Polish	5%	Deployment, tests, accessibility, thoughtful extras

Ground Rules

- **AI tools (ChatGPT, Claude, Copilot, Cursor):** You may use them. We use them too. But you must understand every line of code you submit. We will ask you to walk through your code in the follow-up interview, and using AI as a crutch will be obvious.
- **Be honest:** If you copied a snippet from Stack Overflow or used a tutorial as a starting point, mention it in the README. We respect honesty.
- **Ask if stuck:** If anything in the brief is unclear, email us. We would rather answer a question than receive a submission based on a misunderstanding.

Exercise 1: Personal Task Manager

Best if: This is your first full-stack project, or you want to focus on getting the fundamentals rock-solid.

The Brief

Build a simple task manager (a glorified to-do list) where a user can create, view, update, and delete personal tasks. There is no authentication - assume one user.

Functional Requirements

Must Have

- Add a new task with a title (required) and optional description and due date.
- View all tasks in a list, sorted by creation date (newest first).
- Mark a task as complete or incomplete (toggle).
- Edit a task's title, description, or due date.
- Delete a task, with a confirmation prompt before deletion.
- Filter the list by status: All, Active, Completed.



Should Have

- Show a count of active vs completed tasks somewhere on the screen.
- Visually distinguish overdue tasks (due date is in the past and the task is not complete).
- Empty state UI when there are no tasks.

Nice to Have (Bonus)

- Search tasks by title.
- Persist tasks across server restarts (write to a JSON file or SQLite).
- Drag-and-drop to reorder tasks.

Exercise 2: Mini Expense Tracker

Best if: You want to show off some data handling, basic aggregation, and a slightly richer UI.

The Brief

Build an expense tracker that lets a user log their daily spending across categories and see a summary of where their money is going. No authentication needed - assume one user.

Functional Requirements

Must Have

- Add an expense with: amount (positive number), category (e.g. Food, Transport, Bills, Entertainment, Other), date, and an optional note.
- View all expenses in a table or list, sorted by date (newest first).
- Edit and delete existing expenses.
- Filter expenses by category and by date range (e.g. this month, last month, custom).
- A summary panel showing: total spent this month, total per category, and the highest single expense.

Should Have

- A simple chart showing expenses by category (pie or bar). You can use Recharts, Chart.js, or any library.
- Currency formatting that respects the user's locale, or at least is consistent (e.g. ₹1,234.50).
- Form validation - no negative amounts, no future dates beyond today, category is required.

Nice to Have (Bonus)

- Export visible expenses as a CSV download.
- A small 'budget' setting per category, with a visual indicator when spending exceeds it.
- Persistence to a JSON file or SQLite.

Exercise 3: GitHub Repo Explorer

Best if: You want to show that you can work with a third-party API, handle rate limiting, and think about caching.



The Brief

Build a small web app where a user types a GitHub username, and the app displays that user's public profile along with a list of their public repositories. The frontend should never call the GitHub API directly - your Node.js backend will be the proxy.

Why proxy through the backend? Two reasons we want you to think about: (1) you can cache responses to avoid hitting GitHub's rate limit, and (2) in a real app, you would attach an API token server-side rather than expose it in the browser.

Functional Requirements

Must Have

- Input field where a user enters a GitHub username and hits search.
- Display the user's avatar, name, bio, follower count, following count, and public repo count.
- Display a list of the user's public repositories, with name, description, primary language, star count, and last updated date.
- Sort the repo list by stars, by name, or by last updated - user choice.
- Handle the case where the username does not exist (show a clear error).
- Handle network errors and the GitHub rate limit gracefully.

Should Have

- Server-side caching - if the same username is requested within the last 60 seconds, return the cached response instead of hitting GitHub again. In-memory cache is fine.
- Loading states while the request is in flight (skeletons or spinners).
- Pagination or 'load more' if the user has lots of repos (GitHub returns 30 per page by default).
- Click a repo to expand and see additional details (e.g. open issues count, default branch).

Nice to Have (Bonus)

- A 'recently searched' list that persists in localStorage.
- Show a small chart of repo languages used across the profile.
- Debounced search-as-you-type.

A Few Final Notes

Frequently Asked Questions

Q: Can I use TypeScript? Yes. You will not get extra credit, but you will not be penalised either. Use what you are comfortable with.

Q: Can I use Next.js for both frontend and backend? Yes, that is fine - Next.js API routes count as your Node backend. Just make the structure clear in your README.

Q: Do I need to write tests? Not required, but a couple of meaningful tests on the backend (using Jest or Vitest) will be noticed positively.

Q: How polished should the UI be? Clean, usable, and not broken on mobile. You do not need to be a designer. White space, readable fonts, and clear hierarchy go a long way.

Q: I am stuck on something - can I ask? Yes. Email us. We would much rather clarify than have you guess.

Q: What happens after I submit? We will review your submission within a week, and shortlisted candidates will be called for an offline technical discussion about the exercise and more in the Studio Graphene Gurgaon office.

Good luck - we are looking forward to seeing what you build.